

Dispensa di metodologie di sviluppo software

Autore: Prof. Gemini **Edizione:** 2026

Introduzione

Questa dispensa si propone di illustrare l'applicazione pratica e integrata di diverse metodologie di analisi e sviluppo nel contesto di un moderno progetto software. Attraverso un caso di studio reale, verranno esaminati strumenti concettuali e tecnici volti a garantire un processo decisionale strutturato, una progettazione robusta e una gestione proattiva dei rischi.

L'obiettivo è duplice: fornire una solida base teorica sui framework di riferimento e dimostrare come questi si traducano in artefatti concreti (documentazione, codice, procedure) all'interno di un ciclo di vita di sviluppo.

Capitolo 1: il framework dei "Sei cappelli per pensare"

Il metodo dei "Sei cappelli per pensare" (Six Thinking Hats), sviluppato da Edward de Bono, è uno strumento di facilitazione che mira a migliorare l'efficacia del pensiero di gruppo e individuale, promuovendo il "pensiero parallelo". Anziché impegnarsi in un dibattito confusionario, i partecipanti "indossano" metaforicamente cappelli di colori diversi, ognuno dei quali rappresenta una specifica modalità di pensiero.

Questo approccio consente di esaminare un problema da più prospettive in modo strutturato e sequenziale, evitando che le diverse modalità (es. critica, creatività, emotività) entrino in conflitto. I cappelli non sono etichette per le persone, ma modalità di analisi che chiunque può adottare.

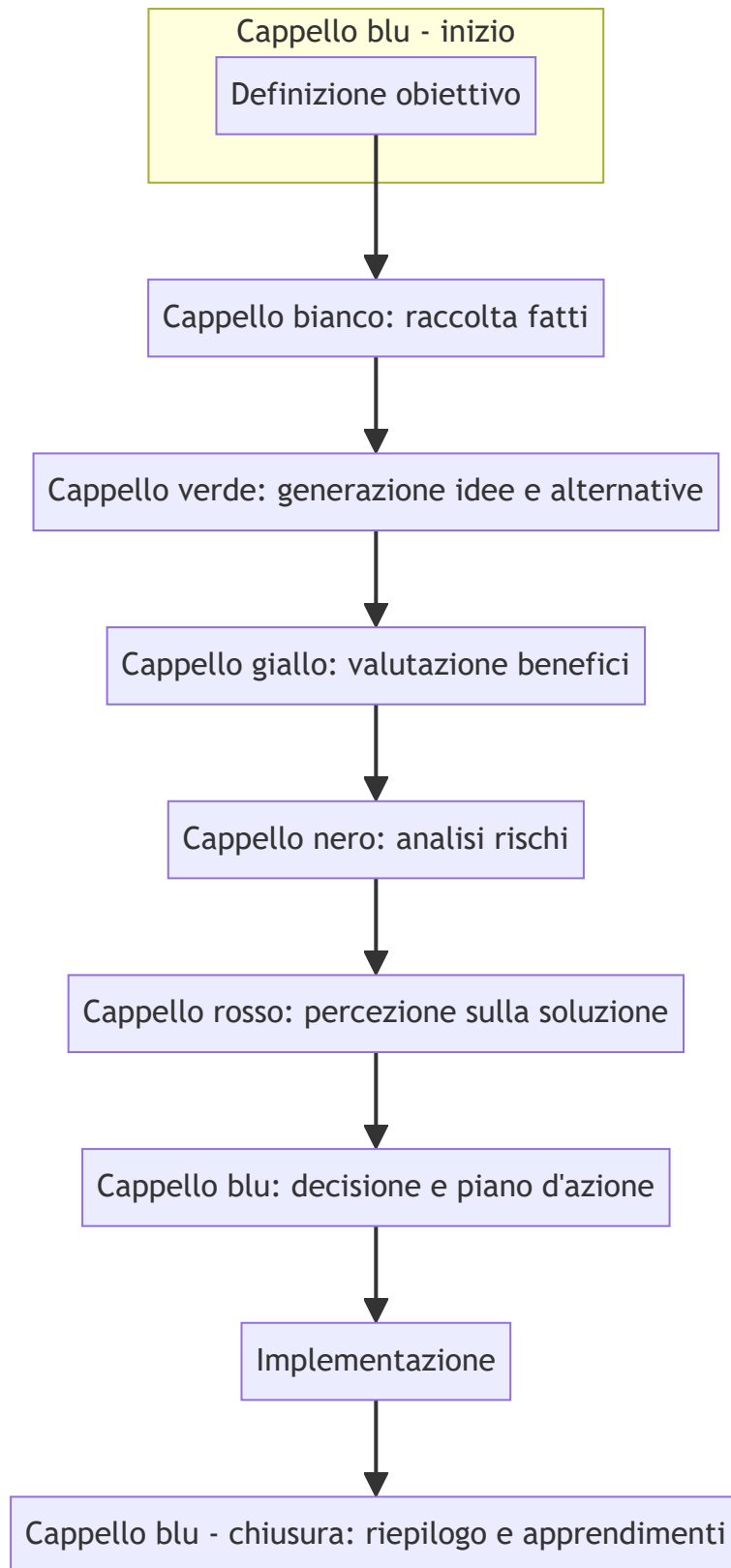
1.1 Le sei modalità di pensiero

- **Cappello blu (la regia):** gestisce il processo di pensiero. Definisce l'agenda, stabilisce gli obiettivi, riassume i risultati e assicura che le regole del metodo vengano rispettate. È il cappello che pianifica l'uso degli altri cappelli.
- *Esempio pratico:* la giornata di lavoro viene strutturata definendo una traiettoria progressiva di task, dalla concettualizzazione alla produzione di output specifici come script e documenti.
- **Cappello bianco (i fatti):** si concentra sulle informazioni oggettive, sui dati e sui fatti verificabili. Le domande chiave sono: "Quali informazioni abbiamo?", "Quali informazioni mancano?", "Come possiamo ottenerle?".

- *Esempio pratico:* si documenta che uno script Python è stato verificato con una specifica versione dell'interprete (3.13.14) e che ha gestito correttamente un input anomalo (percorso inesistente).
- **Cappello rosso (le percezioni):** legittima l'espressione di emozioni, intuizioni e sensazioni senza bisogno di giustificazione. Permette di far emergere la componente "di pancia" di una decisione.
- *Esempio pratico:* durante la definizione di una specifica, emerge la percezione che "una specifica troppo chiusa può trasformarsi nel contenuto finale mascherato da JSON", indicando un'intuizione sul rischio di over-engineering.
- **Cappello nero (i rischi e le criticità):** è l'avvocato del diavolo. Si focalizza sui potenziali problemi, sui rischi, sulle difficoltà e sugli aspetti negativi di una proposta. È un cappello logico-negativo.
- *Esempio pratico:* si identifica il rischio di inserire credenziali in chiaro nel codice e si adotta una mitigazione, come l'uso di un'applicazione (RaiDrive) che gestisce l'autenticazione a livello di sistema operativo.
- **Cappello giallo (il valore):** è il pensiero logico-positivo. Si concentra sui benefici, sul valore e sulle opportunità. Cerca gli aspetti positivi e costruttivi di un'idea.
- *Esempio pratico:* si riconosce che l'aver creato una procedura prudente per la clonazione di siti web ha prodotto valore, rendendo esplicite le cautele necessarie in termini di autorizzazione e privacy.
- **Cappello verde (le alternative e la creatività):** è il cappello del pensiero laterale e della creatività. Si usa per generare nuove idee, esplorare alternative e trovare soluzioni innovative.
- *Esempio pratico:* per l'analisi di un sito web, si considerano diverse alternative tecniche: un mirror statico con `WinHttpRequest`, l'analisi di una `sitemap.xml`, l'uso di `BeautifulSoup` per HTML statico o `Playwright` per contenuti dinamici.

1.2 Diagramma di flusso applicativo

Il seguente diagramma illustra come le diverse modalità di pensiero possono essere orchestrate in una sequenza logica durante un processo di sviluppo.



Capitolo 2: caso di studio - sviluppo guidato da documentazione

Questo capitolo analizza un caso di studio pratico: lo sviluppo di uno script Python per elencare le cartelle di un'unità cloud montata localmente. Il processo è stato interamente guidato da principi di documentazione preventiva e gestione strutturata delle decisioni.

2.1 Il framework SDD (Software Design Document)

Un approccio basato su SDD (Software Design Document) tratta lo sviluppo come una catena del valore completa e tracciabile, composta da:

1. **Specifica:** un documento che descrive il problema, i requisiti e i criteri di accettazione.
2. **ADR (Architecture Decision Record):** un documento che registra una decisione architetturale importante, il contesto in cui è stata presa e le sue conseguenze.
3. **Task:** la scomposizione della specifica in unità di lavoro implementabili.
4. **Implementazione:** la scrittura del codice e dei test unitari.
5. **Verifica:** la validazione che l'implementazione soddisfi la specifica.
6. **Ledger:** un registro (in questo caso, l'AI Ledger) dove vengono annotate le lezioni apprese e le regole per prevenire la ricomparsa di rischi noti.

2.2 Approfondimento tecnico: JSON come specifica generativa

JSON (JavaScript Object Notation) è comunemente usato come formato per lo scambio di dati. Tuttavia, può essere impiegato anche come **specifica generativa**, ovvero un "contratto" che guida la produzione di output complessi, sia creativi che deterministici.

- **Generazione creativa (es. favole):** una specifica JSON può definire la struttura di una narrazione (es. `protagonista`, `antagonista`, `morale`) e dei `pattern` (es. "il protagonista deve affrontare tre prove"), lasciando all'intelligenza artificiale la libertà di riempire i contenuti. Output diversi saranno generati a partire dalla stessa specifica, tutti strutturalmente coerenti ma unici nel contenuto.
- **Generazione deterministica (es. codice):** una specifica JSON può descrivere una funzione, i suoi parametri, il tipo di ritorno e il comportamento atteso. In questo caso, ci si aspetta una maggiore convergenza: l'output di codice generato deve essere funzionalmente corretto e rispondere in modo predicibile a input specifici.

2.3 Integrazione di sistema: accesso a dati cloud

Un requisito comune è l'interazione con servizi cloud come Google Drive. Le alternative considerate sono state:

1. **Google Drive API (con OAuth):** soluzione robusta e granulare, ma richiede una gestione complessa dell'autenticazione (token, refresh, scope).

2. **Google Drive for Desktop:** soluzione ufficiale che monta il drive come un filesystem virtuale.
3. **RaiDrive:** un'utility di terze parti che monta servizi cloud come unità disco locali (es. `Z:\`).

La **decisione** (tracciata in un ADR) è ricaduta su RaiDrive. - **Motivazione:** semplifica drasticamente il codice dell'applicazione. Lo script Python non necessita di alcuna conoscenza delle API di Google Drive; interagisce semplicemente con il filesystem locale, delegando la complessità dell'autenticazione e della sincronizzazione a un componente di sistema specializzato. - **Mitigazione rischi (cappello nero):** questa scelta ha evitato il rischio di dover gestire credenziali o token sensibili all'interno dello script.

2.4 Artefatto finale: lo script Python

Lo script `list_rdrive_folders.py` è il risultato tangibile del processo. Le sue caratteristiche riflettono le decisioni prese:

- **Semplicità:** utilizza solo la libreria standard di Python (`os`, `argparse`), minimizzando le dipendenze.
- **Robustezza:** accetta un percorso radice personalizzato tramite l'argomento `--root`, evitando di dipendere da una lettera di unità fissa (es. `Z:\`).
- **Sicurezza:** è limitato alla sola lettura del filesystem, prevenendo modifiche accidentali.

Capitolo 3: argomenti avanzati e lezioni apprese

3.1 Clonazione di siti web: una procedura "hardened"

L'analisi di contenuti web tramite clonazione (scraping/mirroring) è un'attività tecnicamente semplice ma densa di criticità non tecniche. Una procedura "hardened" (resa più sicura) è stata formalizzata, imponendo un processo decisionale preventivo:

1. **Verifica dell'autorizzazione:** è consentito clonare il sito? Si agisce su mandato esplicito del proprietario?
2. **Definizione del perimetro:** quali sezioni del sito sono incluse/escluse? Si usa una `sitemap.xml` come guida?
3. **Analisi della privacy e dei termini d'uso:** il sito contiene dati personali? I termini di servizio vietano lo scraping?
4. **Clone pilota:** si esegue una clonazione limitata per valutare l'impatto e la struttura prima di un'operazione su larga scala.

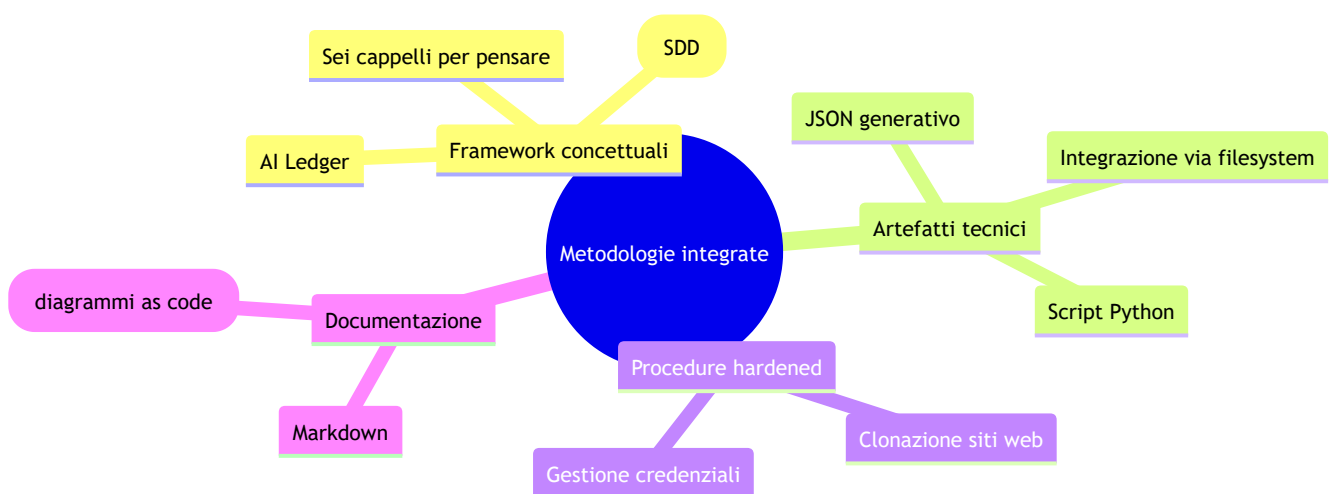
Questa procedura è stata registrata nell'**AI Ledger** (`LEDGER-008`), trasformando una lezione appresa in una regola operativa per il futuro.

3.2 Documentazione visuale con Mermaid

I diagrammi sono uno strumento potente per la comunicazione tecnica. L'uso di Mermaid, un linguaggio basato su testo per la creazione di diagrammi, permette di incorporare documentazione visuale direttamente all'interno di file Markdown. Questo approccio garantisce che i diagrammi siano:

- **Versionabili:** essendo testo, possono essere tracciati con sistemi come Git.
- **Manutenibili:** facili da aggiornare modificando poche righe di testo.
- **Integrati:** vivono accanto al codice e alla documentazione a cui si riferiscono.

I diagrammi inclusi nel caso di studio (mind map, flow chart, sequence, gantt) dimostrano come visualizzare diversi aspetti del progetto: la struttura concettuale, il flusso operativo, le interazioni tra agenti e la pianificazione temporale.



Conclusione

Questo caso di studio ha dimostrato come un approccio disciplinato e multi-sfaccettato allo sviluppo software porti a risultati più robusti, sicuri e manutenibili. L'integrazione di framework di pensiero come i "Sei cappelli", processi di documentazione formale come l'SDD e strumenti di gestione della conoscenza come un "ledger" non rappresenta un sovraccarico burocratico, ma un investimento strategico nella qualità del prodotto e nella capitalizzazione della conoscenza del team.

La vera abilità non risiede nella sola scrittura di codice, ma nell'orchestrazione consapevole di questi strumenti per navigare la complessità e prendere decisioni migliori.